ENGINEERING / VAULTX

One-day task assignments

Smart Contracts / Frontend / Backend

One business day per role | 8 working hours | 07 October 2026

ROLE	ASSIGNMENT	RUNNING RESULT
SC	Reject a zero presale hard cap	Testnet contracts with a deposit demonstrated in the existing frontend.
FE	Prevent empty contact messages	Running contact form showing validation and successful copying.
BE	Serve the six marketplace assets	Hosted API connected to testnet and consumed by the frontend.

ASSIGNMENT RULES

Assign one role per developer. The eight-hour allowance includes setup, implementation, deployment, verification and handover.

SC and BE developers configure and demonstrate integration with the existing frontend. Their submissions contain the relevant contract/backend code and setup notes; frontend source code is not required.

READY BEFORE WORK STARTS

Environment	Working project, testnet RPC, funded test wallet and deployment access.
Smart Contracts	Use the existing presale screen. Resolve frontend setup and ethers compatibility before the assignment.
Backend	Supply a prepared frontend adapter that consumes the API and displays network status. This adapter is absent from the project archive.

REVIEW STANDARD

Submit screenshots from the running implementation together with source code. Screenshots of code alone are insufficient. Missing environments or integration adapters must be reported as blockers. Use test funds only.

SC / ONE BUSINESS DAY

Smart Contracts

Reject a zero presale hard cap

Project file: contracts/VaultXPresale.sol → initialize()

Related files: contracts/VaultXToken.sol, Truffle/migrations/2_deploy_contracts.js, src/shared/config/contracts.js

The hard cap is the maximum amount the sale can accept. Reject zero when the owner initializes the sale.

IMPLEMENTATION

1. Add one check: when _hardCap is zero, revert with InvalidConfiguration("zero hard cap").
2. Place it before the existing soft-cap comparison and token metadata reads. Keep the other rules unchanged.
3. Check zero-cap rejection, initialized=false after rejection, and valid initialization.
4. Deploy the updated presale and VaultXToken to the team's testnet. Follow the existing deployment order: presale first, token second, initialize last. Route the token's presale allocation to the new presale address.
5. Configure the existing frontend with the deployed addresses and connect a wallet on the same testnet. Use an active sale window and a small test contribution agreed with the team.
6. Make one deposit through the presale screen. Confirm the receipt and refreshed buyer allocation.

ACCEPTANCE CRITERIA

- A zero hard cap is rejected; valid settings still initialize.
- Both deployed addresses and transaction hashes are available on the testnet explorer.
- The running frontend reads the new deployment and completes one test deposit.

SUBMISSION | RUNNING SCREENSHOTS + SOURCE CODE

Code	changed contract and deployment/test files. Frontend source code is not required.
Screenshots	zero-cap rejection and initialized=false; explorer deployment evidence; frontend with testnet and sale data; confirmed deposit and refreshed allocation.
Handover	testnet name/chain ID, contract addresses, transaction hashes, frontend URL, configuration variable names and run/check commands. Do not include private keys.

TIME ALLOCATION / 8 HOURS

setup 1h · change/checks 2h · deployment 2h · frontend demonstration 2h · handover 1h.

FE / ONE BUSINESS DAY

Frontend

Stop empty contact messages

Project file: src/pages/contact/index.jsx

Functions: prepareSummary() and the review-request textarea

The page copies an inquiry to the clipboard. Stop copying when the review message is empty.

IMPLEMENTATION

1. On submit, check message.trim().
2. If empty, show "Please enter a review message." and do not call the clipboard API.
3. Clear the error when the user edits the message.
4. Preserve the existing copy-summary behavior for valid messages and the current layout.

ACCEPTANCE CRITERIA

- Empty and spaces-only input show the error and copy nothing.
- Editing clears the error.
- A normal message copies the existing summary.

SUBMISSION | RUNNING SCREENSHOTS + SOURCE CODE

Code	changed contact-page file.
Screenshots	the running app's empty-message error, then a valid message and copy-success state. Keep the browser URL visible.
Handover	run commands, a three-case checklist and npm run build result. Use localhost for the clipboard checks. This task does not require a blockchain transaction.

TIME ALLOCATION / 8 HOURS

setup 2h · change 2h · browser checks 2h · screenshots/notes 1h · buffer 1h.

BE / ONE BUSINESS DAY

Backend

Serve project assets and testnet status

Data source: src/features/marketplace/GalleryItems.jsx → DATA

New files: backend/server.mjs and a short backend README

The existing marketplace contains six sample assets. Return those assets through a small API and prove the deployed service connects to the team's testnet.

IMPLEMENTATION

1. Copy id, name, location and type from all six existing assets.
2. Add GET /api/assets returning 200 JSON {"assets":[...]}.
3. Add GET /api/network. Read eth_chainId and eth_blockNumber from TESTNET_RPC_URL; return {"chainId":"0x...","blockNumber":"0x..."} using actual RPC results.
4. Return 502 JSON {"error":"Testnet unavailable"} if the RPC fails. Return 404 JSON {"error":"Not found"} for unknown paths. Set JSON Content-Type.
5. Deploy the API to the team's hosting environment. Keep the RPC URL in server environment configuration. A backend service is hosted off-chain; it connects to testnet rather than being deployed as a smart contract.
6. Configure the team-supplied frontend adapter with the API URL. Show all six assets and the actual testnet chain/block status in the running frontend. Allow the frontend origin if cross-origin requests are needed.

ACCEPTANCE CRITERIA

- The deployed assets endpoint returns the six project assets.
- Network status comes from the configured testnet RPC.
- The running frontend consumes the deployed API and displays assets plus network status.
- Unknown routes return 404; RPC failure returns 502.

SUBMISSION | RUNNING SCREENSHOTS + SOURCE CODE

Code	backend implementation and deployment configuration. Frontend source code is not required.
Screenshots	deployed API responses with URLs/status codes; the running frontend displaying assets and testnet status; one error-response check.
Handover	API URL, frontend URL, testnet name/chain ID, environment variable names and run/check commands. No database, authentication or POST route is required.

TIME ALLOCATION / 8 HOURS

setup 1h · endpoints 3h · deployment 1h · frontend demonstration 2h · handover 1h.